Gossip-Based Peer-to-Peer Network with Consensus-Driven Membership

Computer Networks Assignment - 1

Member 1: *Aashcharya Gorakh*

Roll No.: *B23ES1001*

Member 2: *Namya Dhingra*

Roll No.: *B23CS1040*

Language: C++17 (Winsock2)

Date: 26 February 2026

Department of Computer Science & Engineering

Indian Institute of Technology, Jodhpur

Contents

1 Introduction and Objective **3**

2 System Architecture **3**
2.1 High-Level Overview . . . 3
2.2 System Startup Sequence . . . 4
2.3 Node Types . . . 5

3 Project Structure and File Organization **6**

4 Communication Protocol **6**
4.1 Message Format . . . 6
4.2 Message Types . . . 7

5 Detailed Design and Implementation **7**
5.1 Seed Node (`seed.cpp`) . . . 7
5.1.1 Data Structures . . . 7
5.1.2 Registration Consensus . . . 8
5.1.3 Removal Consensus . . . 8
5.1.4 Full Message Lifecycle — `handle_client()` . . . 8
5.2 Peer Node (`peer.cpp`) . . . 9
5.2.1 Full Message Lifecycle — `handle_peer_client()` . . . 10
5.2.2 Dynamic Preferential Attachment Overlay . . . 11
5.3 Gossip Protocol . . . 12
5.3.1 Message Generation . . . 12
5.3.2 Forwarding and Deduplication . . . 12
5.4 Failure Detection — Two-Level Consensus . . . 13
5.4.1 Level 1: Peer-Level Consensus . . . 13
5.4.2 Level 2: Seed-Level Consensus . . . 13
5.4.3 Seed-Level Eviction Consensus — Sequence Diagram . . . 14

6 Logging and Output **15**

7 Security Analysis **16**
7.1 Threat Model and Mitigations . . . 17
7.2 Analysis . . . 17

8 Testing **17**
8.1 Test Procedure . . . 18
8.2 How to Run Tests . . . 18
8.3 Console Output Screenshots . . . 20

9 How to Compile and Run **21**
9.1 Prerequisites . . . 21
9.2 Quick Start (Automated) . . . 21
9.3 Manual Compilation . . . 21
9.4 Starting Individual Nodes . . . 22
9.5 Network Replay Visualizer . . . 22
9.6 Output Files . . . 22

10 GUI and Visualization **22**
10.1 Live Network Dashboard (`visualizer.html`) 22

11 Sample Log Output **23**

12 Design Decisions and Trade-offs **25**

13 Potential Improvements **26**

14 Conclusion **26**

References **27**

1 Introduction and Objective

The objective of this project is to design and implement a **gossip-based peer-to-peer (P2P) network** that supports:

- **Reliable message dissemination** via epidemic (gossip) forwarding.
- **Robust liveness detection** using TCP-level socket checks with multi-peer confirmation.
- **Consensus-driven membership management** preventing unilateral node addition or removal.

The system employs a **two-level consensus** architecture: peer-level agreement among neighbors confirms suspected failures, and seed-level quorum ($\lfloor n/2 \rfloor + 1$ out of n seeds) authorizes all membership changes. This design mitigates Sybil-style attacks and false accusations by ensuring that no single node can unilaterally modify the network's membership.

The entire implementation is written in **C++17** using the **Windows Sockets 2 (Winsock2)** API for network communication, `std::thread` for concurrency, and `std::mutex` for thread-safe shared-state access. A pure C++ SHA-256 implementation provides cryptographic message hashing with zero external dependencies.

Key Design Principle

Every membership change—registration or removal—requires agreement from a majority of seed nodes. Dead-node detection additionally requires peer-level consensus before escalation to seeds.

2 System Architecture

2.1 High-Level Overview

The network is organized into two planes:

1. **Control Plane** — Seed nodes that serve as the distributed membership authority.
2. **Data Plane** — Peer nodes that form a gossip overlay for message dissemination.

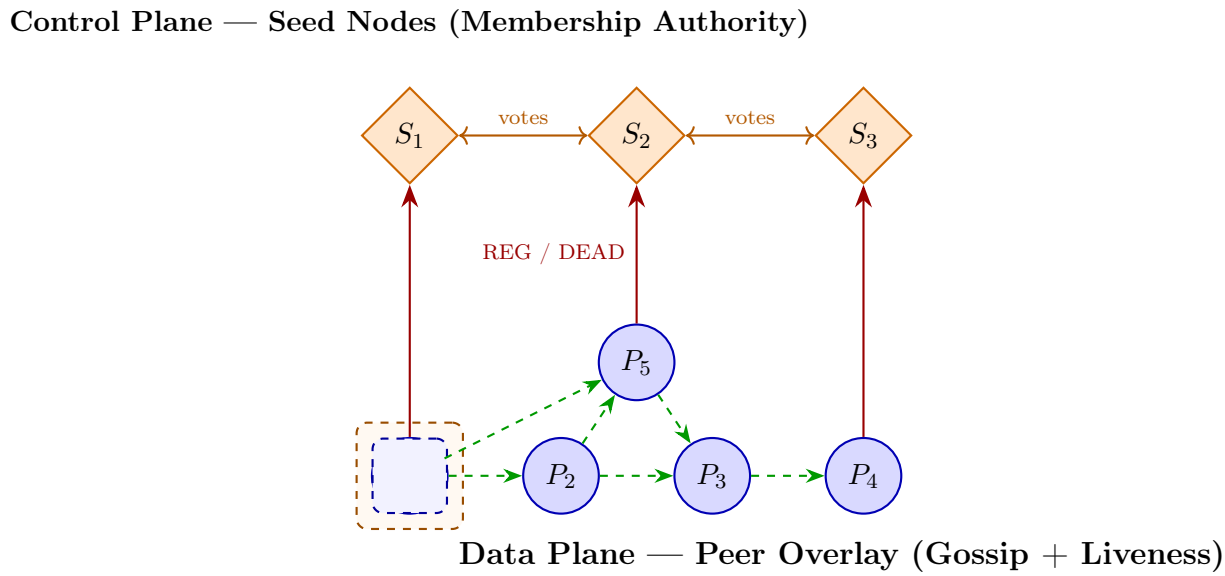


Figure 1: Two-plane architecture: Seed nodes form the control plane (consensus-based membership); Peer nodes form the data plane (gossip overlay with power-law degree distribution).

2.2 System Startup Sequence

Figure 2 shows the full startup sequence diagram: `run_all.py` compiles both binaries, spawns 3 seed processes (each initializing Winsock, loading `config.txt`, and binding their listener sockets), waits for seeds to be ready, then spawns 3 peer processes that register, build topology, and begin gossip/liveness threads.

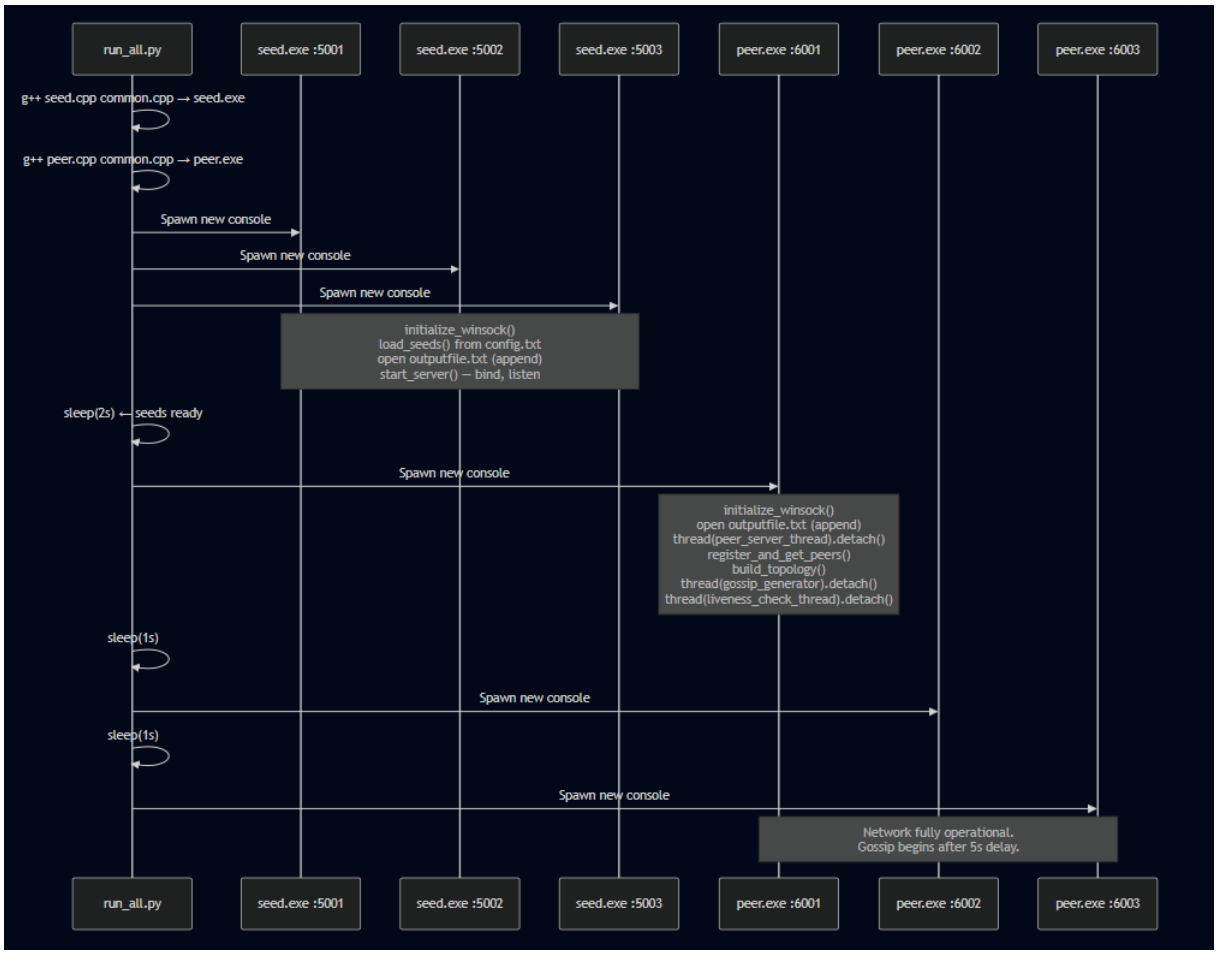


Figure 2: System startup sequence: `run_all.py` orchestrates compilation and process deployment. Seeds initialize first, then peers register and build the overlay.

2.3 Node Types

Table 1: Comparison of Seed and Peer node responsibilities.

Aspect	**Seed Node**	**Peer Node**
Role	Membership authority	Data-plane participant
Gossip	Does NOT participate	Generates & forwards
Peer List	Maintains global PL	Maintains neighbor list
Registration	Proposes & votes	Sends request, awaits quorum
Removal	Votes on dead-node proposals	Detects failure, reports
Consensus	Seed-level quorum	Peer-level majority
Connections	Accepts from peers & seeds	Connects to seeds & peers

3 Project Structure and File Organization

Table 2: Project file organization with line counts and responsibilities.

File	Lines	Purpose
`config.txt`	Auto	Seed node addresses (`IP:Port` per line)
`common.h`	41	Header: Winsock2 includes, socket helpers, SHA-256, `get_timestamp()` declarations
`common.cpp`	166	Implementation: Winsock init/cleanup, `send_json()`/`recv_json()`, pure C++ SHA-256, millisecond-precision timestamps
`seed.cpp`	618	Seed node: registration/removal consensus, PL management, seed-to-seed sync
`peer.cpp`	628	Peer node: preferential attachment overlay, gossip engine, liveness detection, dead-node reporting
`run_all.py`	113	Python automation: compiles both binaries, launches 3 seeds + 3 peers in separate terminals, interactive kill/list console
`visualizer.html`	1176	Live JavaScript dashboard (canvas topology, event feed, stats, playback controls)
`outputfile.txt`	Auto	Combined log output from all nodes (spec requirement)

4 Communication Protocol

4.1 Message Format

All inter-node communication uses **TCP sockets** (Winsock2) with **custom string-encoded messages**. Each message is a single string sent atomically via `send_json()`, which loops until all bytes are written, and received via `recv_json()` into a fixed 4096-byte buffer.

```
// Registration request (Peer -> Seed):
"TYPE:REGISTER;PEER:127.0.0.1:6001"

// Gossip message (Peer -> Peer):
"TYPE:GOSSIP;MSG:1772104070:127.0.0.1:6001:0;SENDER:127.0.0.1:6001"

// Dead node report (Peer -> Seed):
"Dead Node:127.0.0.1:6001:1772104317:127.0.0.1"
```

Listing 1: Message structure (custom string protocol)

4.2 Message Types

Table 3: Complete list of protocol message types.

Direction	Type	Purpose
Peer → Seed	`TYPE:REGISTER`	Request to join the network
	`Dead Node:...`	Report a confirmed dead peer
	`TYPE:GET_PEERS`	Request current peer list
Seed → Peer	`TYPE:PEER_LIST`	Response with peer list
Seed ↔ Seed	`TYPE:VOTE`	Relay registration vote to other seeds
	`TYPE:SEED_DEAD_VOTE`	Relay dead-node vote to other seeds
	`TYPE:SYNC_LIST`	Periodic peer list synchronization
Peer ↔ Peer	`TYPE:GOSSIP`	Gossip message forwarding
	`TYPE:CONNECT_REQ`	Bidirectional topology link request
	`TYPE:GET_DEGREE`	Query neighbor count (for preferential attachment)
	`TYPE:DEGREE_REPLY`	Response with current degree
	`TYPE:SUSPICION`	Ask neighbor about suspect
	`TYPE:SUSPICION_CONFIRM`	Confirm suspicion

5 Detailed Design and Implementation

5.1 Seed Node (`seed.cpp`)

The seed node is implemented as a multi-threaded TCP server using Winsock2, with three concurrent threads:

1. **Listener thread** — accepts incoming TCP connections and dispatches each to `handle_client()` in a detached `std::thread`.
2. **Sync thread** — periodically (every 30 s) exchanges peer lists with other seeds via `TYPE:SYNC_LIST` messages to ensure eventual consistency.
3. **Main thread** — initializes Winsock, loads config, opens the log file, and enters the accept loop.

5.1.1 Data Structures

- `peer_list` (`std::set<string>`): The consensus-approved set of live peers (`IP:PORT`). Protected by a global `std::mutex mtx`.
- `votes` (`std::map<string, std::set<int>>`): Maps each pending peer to the set of seed ports that have voted for its registration.
- `dead_reports` (`std::map<string, std::set<int>>`): Maps each suspect peer to the set of seed ports that have reported it dead.

5.1.2 Registration Consensus

Algorithm 1 Consensus-Based Peer Registration

```
procedure OnRegisterRequest(peer_id)
    if peer_id already in Peer List then
        Skip (already registered)                    ▷ Avoid duplicates
        return
    end if
    Cast own vote: votes[peer_id].insert(self_port)
    Broadcast TYPE:VOTE to all other seeds
    for each arriving TYPE:VOTE for peer_id do
        Record vote in votes[peer_id]
    end for
    vote_count ← |votes[peer_id]|
    if vote_count ≥ ⌊n/2⌋ + 1 then
        Add peer to Peer List
        Log: "CONSENSUS OUTCOME – Peer added: peer_id"
    else
        Log: "CONSENSUS PROGRESS – votes so far"
    end if
end procedure
```

5.1.3 Removal Consensus

The removal consensus follows an identical pattern: a seed receiving a `Dead Node:` report records its own dead vote, broadcasts a `TYPE:SEED_DEAD_VOTE` to other seeds, and removes the peer from the PL only upon quorum approval. The `dead_reports` entry is cleaned up after removal to permit re-registration.

5.1.4 Full Message Lifecycle — `handle_client()`

Figure 3 shows the complete decision tree inside the seed's `handle_client()` function. Each incoming TCP connection is dispatched here; the handler parses the message type and routes to the appropriate logic (registration vote, dead-node vote, peer list response, or sync).

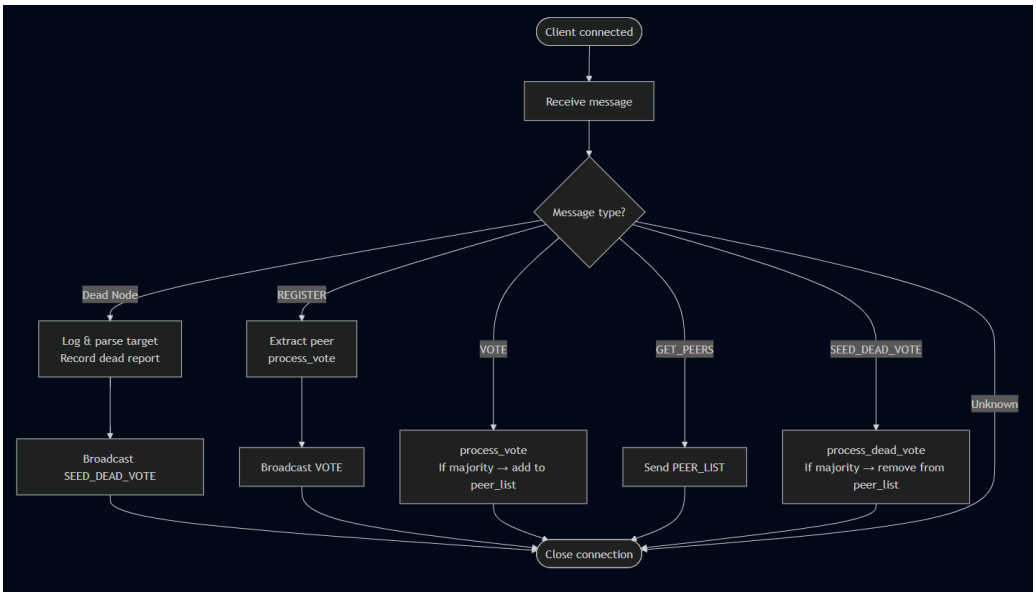


Figure 3: Full message lifecycle flowchart for `seed.cpp - handle_client()`: incoming connections are routed based on message type to the appropriate handler (REGISTER, VOTE, GET_PEERS, SEED_DEAD_VOTE, Dead Node, SYNC_LIST).

5.2 Peer Node (`peer.cpp`)

On startup, each peer executes the following lifecycle:

1. **Registration**: Connect to $\lfloor n/2 \rfloor + 1$ randomly chosen seeds (using `std::shuffle`) and send `TYPE:REGISTER`.
2. **Peer List Fetch**: Request peer lists from the chosen seeds via `TYPE:GET_PEERS` and compute their union.
3. **Overlay Construction**: Select neighbors using **dynamic preferential attachment** — query each peer's live degree and use weighted random sampling.
4. **Gossip Loop**: Generate up to 10 messages (one every 5 s), forward to all neighbors.
5. **Liveness Loop**: Periodically TCP-probe neighbors; initiate peer-level consensus on suspects.

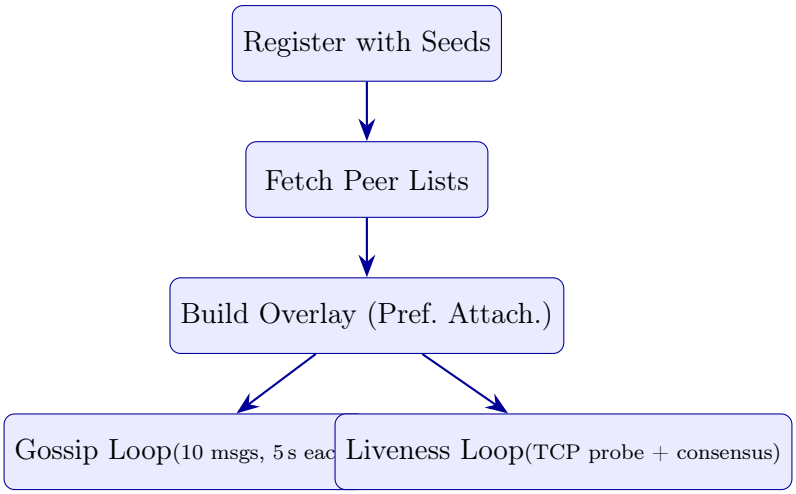


Figure 4: Peer node lifecycle: sequential initialization followed by concurrent gossip and liveness threads.

5.2.1 Full Message Lifecycle — `handle_peer_client()`

Figure 5 shows the complete decision tree for the peer's incoming message handler. It processes degree queries, connection requests, gossip (with SHA-256 dedup), suspicion queries, and suspicion confirmations.

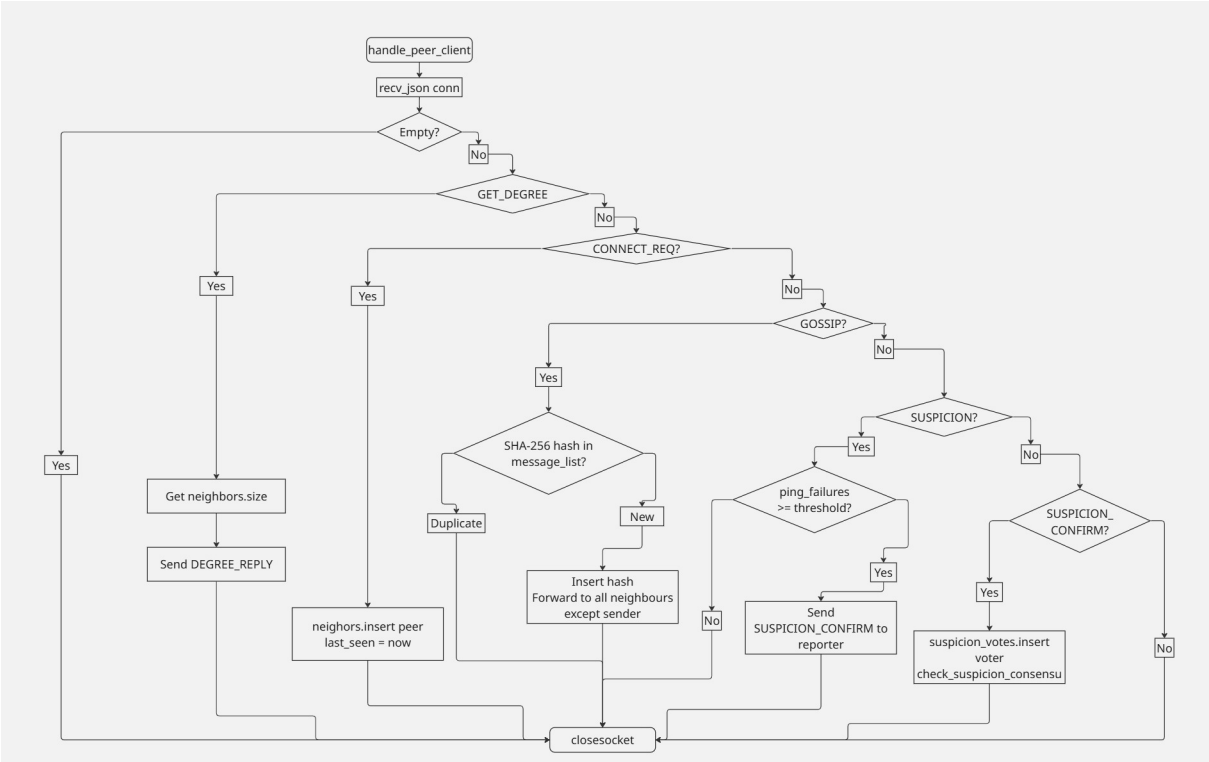


Figure 5: Full message lifecycle flowchart for `peer.cpp - handle_peer_client()`: incoming messages are parsed and routed to handlers for GET_DEGREE, CONNECT_REQ, GOSSIP (with SHA-256 dedup check), SUSPICION, and SUSPICION_CONFIRM.

5.2.2 Dynamic Preferential Attachment Overlay

The overlay topology follows a **power-law degree distribution** using **dynamic preferential attachment**. Unlike static Zipf-weighted sampling, our approach queries each candidate peer's *actual current degree* over the network via `TYPE:GET_DEGREE` / `TYPE:DEGREE_REPLY` messages, then uses `std::discrete_distribution` for weighted selection.

Given a set of available peers, each peer k receives weight:

$$w(k) = \text{degree}(k) + 1$$

The +1 base weight ensures that zero-degree nodes still have a non-zero probability of being selected, preventing starvation. The probability of selecting peer k as a neighbor is:

$$P(k) = \frac{w(k)}{\sum_{j=1}^{N} w(j)} = \frac{\text{degree}(k) + 1}{\sum_{j=1}^{N} (\text{degree}(j) + 1)}$$

This naturally produces a "rich-get-richer" dynamic: peers with more existing connections are preferentially chosen as neighbors, creating hub nodes characteristic of real-world scale-free networks (Barabási–Albert model).

```cpp
vector<int> weights;
for (auto& peer : available) {
    // Query actual live degree from the network
    int peer_degree = 0;
    SOCKET s = socket(AF_INET, SOCK_STREAM, 0);
    // ... connect to peer ...
    send_json(s, "TYPE:GET_DEGREE");
    string res = recv_json(s);
    if (res.find("DEG:") != string::npos)
        peer_degree = stoi(res.substr(res.find("DEG:") + 4));
    closesocket(s);
    weights.push_back(peer_degree + 1); // Base weight
}

int num = max(1, (int)available.size() / 2 + 1);
random_device rd;
mt19937 gen(rd());

for (int i = 0; i < num && !available.empty(); i++) {
    discrete_distribution<int> dist(weights.begin(), weights.end());
    int idx = dist(gen);
    // Connect to available[idx], establish bidirectional link
    send_json(s, "TYPE:CONNECT_REQ;PEER:" + SELF_ID);
    neighbors.insert(available[idx]);
    available.erase(available.begin() + idx);
    weights.erase(weights.begin() + idx);
}
```

Listing 2: Dynamic preferential attachment (peer.cpp, simplified)

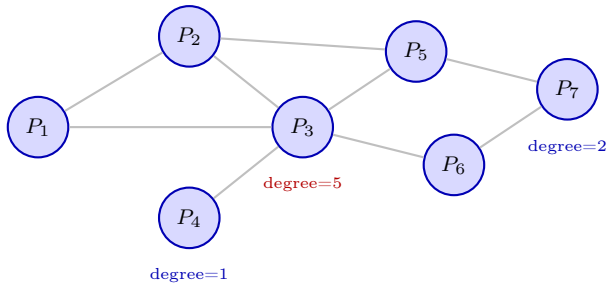


Figure 6: Example power-law overlay: P_3 acts as a hub (high degree), while P_4 and P_7 have fewer connections. Dynamic preferential attachment amplifies this heterogeneity, ensuring efficient gossip propagation in $O(\log N)$ hops.

5.3 Gossip Protocol

5.3.1 Message Generation

Each peer generates gossip messages at 5-second intervals, up to a maximum of 10 messages. The gossip message format is:

```
<timestamp>:<IP>:<Port>:<Msg#>
```

Example: `1772104070:127.0.0.1:6001:0`

5.3.2 Forwarding and Deduplication

Algorithm 2 Gossip Forwarding with SHA-256 Deduplication

```
procedure OnGossipReceived(M, sender)
    h ← SHA-256(M.msg_id)                    ▷ Pure C++ implementation
    if h ∈ MessageList then
        return                               ▷ Duplicate — silently drop
    end if
    MessageList.insert(h)
    Log: "GOSSIP RECEIVED M.id | From: sender"
    for each neighbor n ≠ sender do
        Forward M to n via TCP
    end for
end procedure
```

This ensures each message traverses each link **at most once**, preventing loops and redundant forwarding. The SHA-256 implementation is a pure C++ port requiring no external libraries (OpenSSL, Boost, etc.), keeping the project dependency-free.

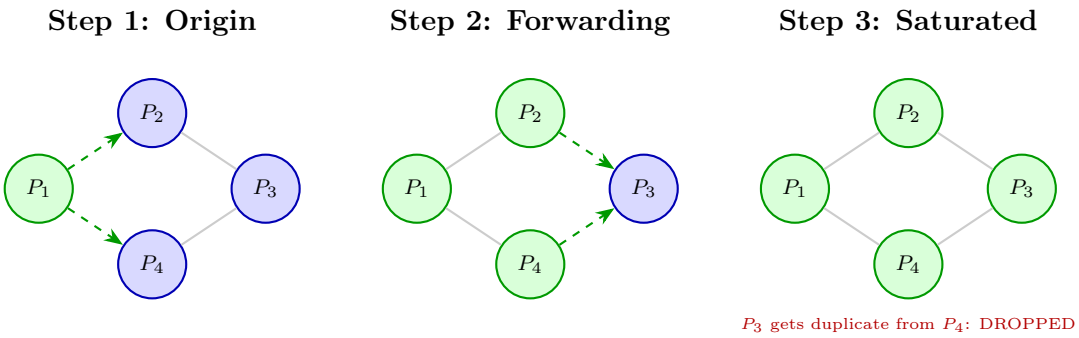


Figure 7: Gossip propagation: P_1 originates a message, forwards to neighbors. Each peer records the SHA-256 hash and forwards once. Duplicates are silently dropped.

5.4 Failure Detection — Two-Level Consensus

Dead-node detection uses a **two-level consensus** protocol to prevent false accusations:

5.4.1 Level 1: Peer-Level Consensus

1. Peer P_i periodically performs a TCP `connect()` probe to each neighbor's port (every 5 s).
2. If the connection is refused (process dead), `ping_failures[peer]++`.
3. After the counter reaches threshold (`PING_FAIL_THRESHOLD = 3`), P_i enters the **suspicion phase**.
4. P_i sends `TYPE:SUSPICION` to all other neighbors asking if they also find the suspect unresponsive.
5. Each queried peer checks its own `ping_failures` for the suspect and, if it also exceeds the threshold, replies with `TYPE:SUSPICION_CONFIRM`.
6. If a **majority of neighbors** confirm ($\geq \lfloor k/2 \rfloor + 1$ out of k neighbors), peer-level consensus is reached.

5.4.2 Level 2: Seed-Level Consensus

1. P_i sends a `Dead Node:` report to **all** seeds with the format:

```
Dead Node:<IP>:<Port>:<timestamp>:<reporter_IP>
```

2. The receiving seed records its own dead vote and broadcasts `TYPE:SEED_DEAD_VOTE` to other seeds.
3. If $\geq \lfloor n/2 \rfloor + 1$ seeds record the dead vote, the peer is removed from all peer lists.

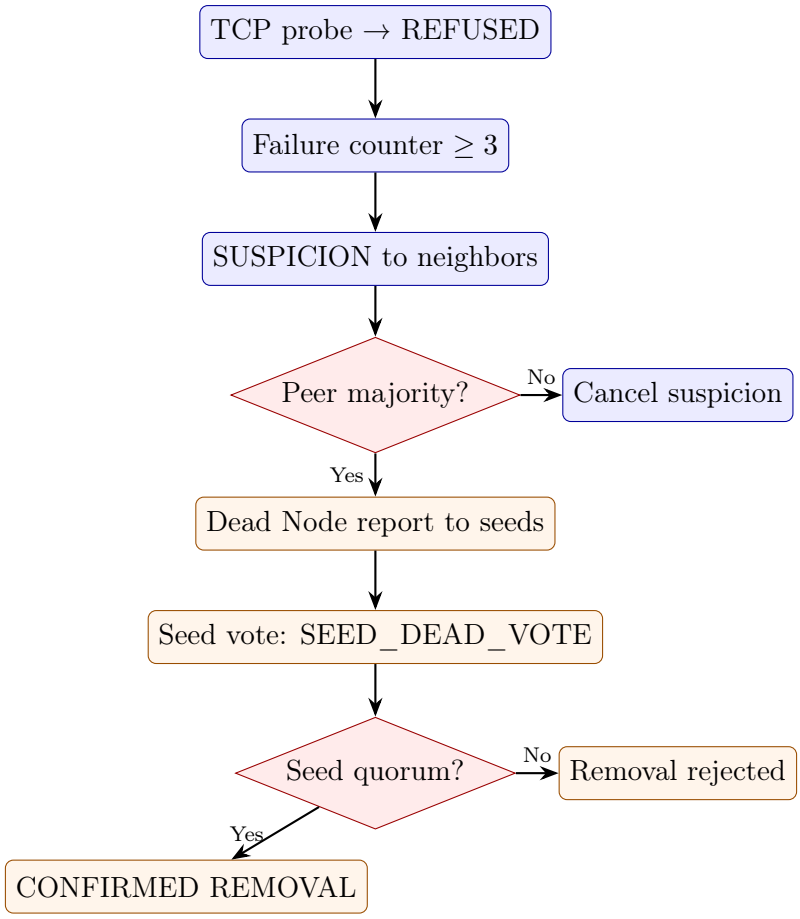


Figure 8: Two-level consensus flow for dead-node detection and removal. Both majority agreements must pass before any membership change takes effect.

5.4.3 Seed-Level Eviction Consensus — Sequence Diagram

Figure 9 illustrates a concrete example of the seed-level dead-node eviction protocol. After a peer reports `127.0.0.1:6001` as dead, the receiving seeds exchange `SEED_DEAD_VOTE` messages until quorum is reached and the peer is removed from all peer lists.

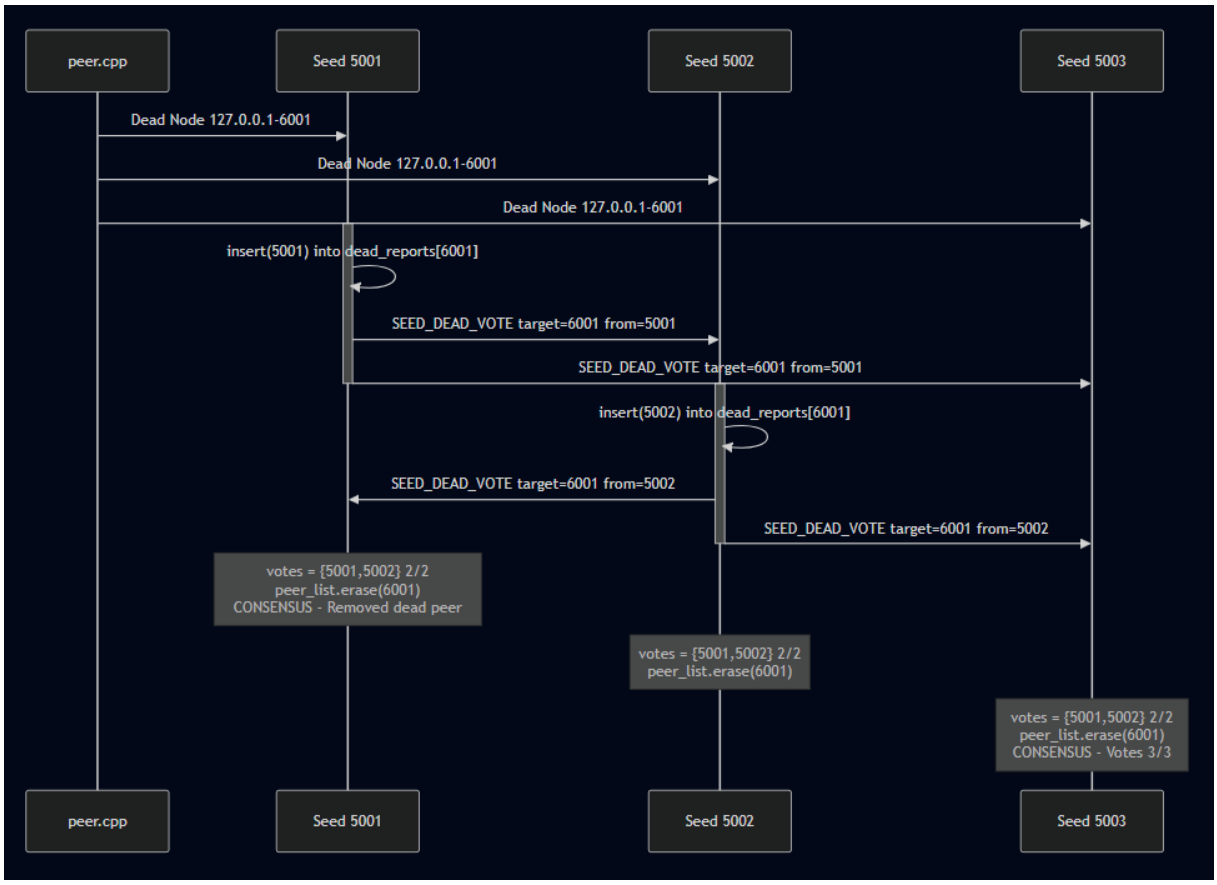


Figure 9: Seed-level dead-node eviction consensus: peer.cpp sends the dead-node report to all 3 seeds. Seed 5001 records the report, broadcasts `SEED_DEAD_VOTE` to 5002 and 5003. Upon quorum (2/3), each seed removes the peer from its list.

6 Logging and Output

The logging system writes to two destinations simultaneously:

1. **Console** — real-time output for monitoring individual node terminals.
2. `outputfile.txt` — combined log in the project root (spec requirement). Opened in append mode (`ios::app`) so logs from all concurrently running nodes interleave into a single chronological file.

Each log line follows the format:

```
YYYY-MM-DD HH:MM:SS.mmm [LEVEL] [IP:PORT] message
```

Example output:

```
2026-02-26 14:30:12.456 [INFO] [127.0.0.1:5001] [CONSENSUS OUTCOME]
    Peer added: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 14:30:18.789 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED]
    1772104070:127.0.0.1:6001:0 | From: 127.0.0.1:6001
```

Timestamps include millisecond precision via `std::chrono::system_clock`, and log writes are immediately flushed (`log_file.flush()`) to prevent data loss on crash.

Table 4: Logged events per node type, as required by the spec.

Node	**Event**	**Log Level**
Seed	Registration proposals (with vote counts)	INFO
Seed	Consensus outcomes (peer added)	INFO
Seed	Confirmed dead-node removals	WARNING
Seed	Current Peer List after changes	INFO
Seed	Periodic sync events	INFO
Peer	Received Peer Lists from seeds	INFO
Peer	Gossip generated / received (timestamp, sender)	INFO
Peer	Topology connections and neighbor list	INFO
Peer	Suspicion broadcasts and votes	WARNING
Peer	Confirmed dead-node reports	WARNING
Both	TCP liveness probes (PING/PONG equivalent)	*NOT logged*

7 Security Analysis

The two-level consensus mechanism provides strong mitigation against several attack vectors:

7.1 Threat Model and Mitigations

Table 5: Security threats and consensus-based mitigations.

Attack	Description	Mitigation
False Accusation	Malicious peer falsely reports a healthy node as dead	Peer-level majority required before escalation; seeds independently verify via quorum
Sybil Attack	Attacker tries to register bogus nodes to gain influence	Seed consensus ($\lfloor n/2 \rfloor + 1$ votes) prevents unilateral registration
Collusion	Multiple peers collude to evict a target	Requires corrupting $> 50\%$ of the target's neighbors AND $> 50\%$ of seeds simultaneously
Unilateral Removal	Single seed removes a peer	Impossible—removal requires seed-level quorum
Replay Attack	Replaying old dead-node reports	Timestamp-based detection embedded in the dead-node report format
Idempotency Abuse	Repeated registration of same peer	Duplicate votes silently ignored (set-based vote tracking)

7.2 Analysis

- **False accusation resistance**: A single peer cannot remove any other node. At minimum, $\lfloor k/2 \rfloor + 1$ neighbors must independently confirm the peer is unresponsive via TCP connection probes, AND subsequently $\lfloor n/2 \rfloor + 1$ seeds must approve the removal.
- **Byzantine tolerance**: The system tolerates up to $\lfloor n/2 \rfloor$ compromised seeds without losing consensus correctness. For peer-level consensus, tolerance is $\lfloor k/2 \rfloor$ compromised neighbors per target node.
- **Practical limitation**: The system assumes a non-Byzantine majority at both levels. If $> 50\%$ of seeds are compromised, arbitrary membership changes become possible. This is a fundamental limitation of majority-vote consensus.

8 Testing

Testing is performed via the `run_all.py` automation script, which compiles both binaries, launches 3 seeds and 3 peers in separate terminal windows, and provides an interactive console for killing nodes to trigger fault-tolerance logic.

8.1 Test Procedure

Table 6: Manual test cases validated against the specification.

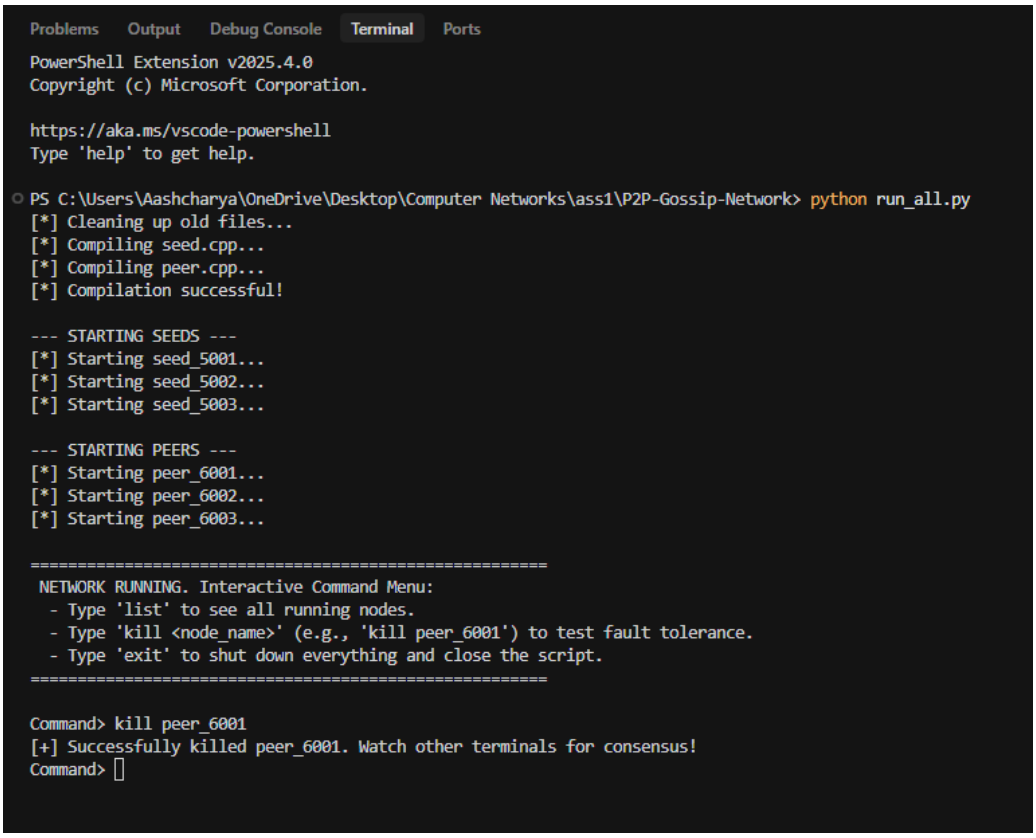
Stage	#	Test Case	Result
1	1	Seeds start and accept TCP connections on configured ports	PASS
	2	Seeds respond to `GET_PEERS` with `PEER_LIST`	PASS
2	3	Peer registration achieves seed consensus ($\geq 2/3$ votes)	PASS
	4	Registered peer appears in Peer List on all seeds	PASS
	5	Multiple peers register via different seed subsets	PASS
3	6	Preferential attachment overlay builds with bidirectional links	PASS
	7	Degree queries (`GET_DEGREE`) return correct counts	PASS
4	8	Gossip messages propagate to all connected peers	PASS
	9	Duplicate gossip (same SHA-256 hash) silently dropped	PASS
	10	10 messages per peer, 5 s apart	PASS
5	11	Killed peer detected after 3 consecutive TCP probe failures	PASS
	12	Peer-level suspicion consensus achieved	PASS
	13	Seed-level removal consensus removes dead peer from PL	PASS
6	14	Seed-to-seed periodic sync ensures PL consistency	PASS

8.2 How to Run Tests

```python
# Start entire network (compiles, launches 3 seeds + 3 peers)
python run_all.py

# Interactive commands in the script console:
#   list            - show all running nodes
#   kill peer_6001  - terminate a peer to test fault tolerance
#   exit            - shutdown everything
```

Listing 3: Running the network via automation script

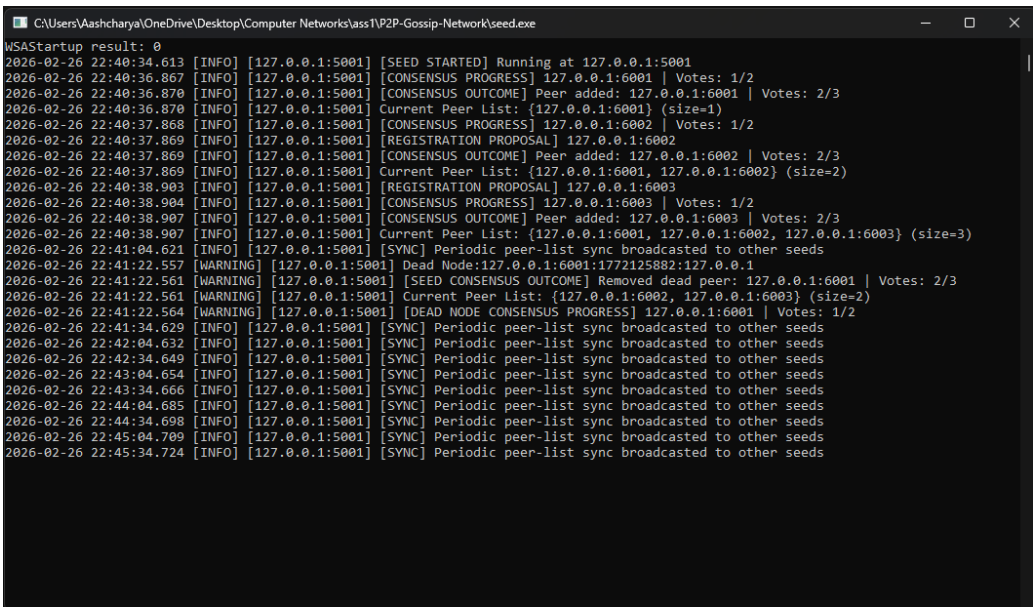
The `run_all.py` script automatically compiles `seed.cpp` and `peer.cpp` with `g++`, starts 6 processes in separate terminal windows (using `CREATE_NEW_CONSOLE`), and provides real-time control for simulating node failures.



```
Problems   Output   Debug Console   Terminal   Ports
PowerShell Extension v2025.4.0
Copyright (c) Microsoft Corporation.

https://aka.ms/vscode-powershell
Type 'help' to get help.

PS C:\Users\Aashcharya\OneDrive\Desktop\Computer Networks\ass1\P2P-Gossip-Network> python run_all.py
[*] Cleaning up old files...
[*] Compiling seed.cpp...
[*] Compiling peer.cpp...
[*] Compilation successful!

--- STARTING SEEDS ---
[*] Starting seed_5001...
[*] Starting seed_5002...
[*] Starting seed_5003...

--- STARTING PEERS ---
[*] Starting peer_6001...
[*] Starting peer_6002...
[*] Starting peer_6003...

====================================================
 NETWORK RUNNING. Interactive Command Menu:
  - Type 'list' to see all running nodes.
  - Type 'kill <node_name>' (e.g., 'kill peer_6001') to test fault tolerance.
  - Type 'exit' to shut down everything and close the script.
====================================================

Command> kill peer_6001
[+] Successfully killed peer_6001. Watch other terminals for consensus!
Command>
```

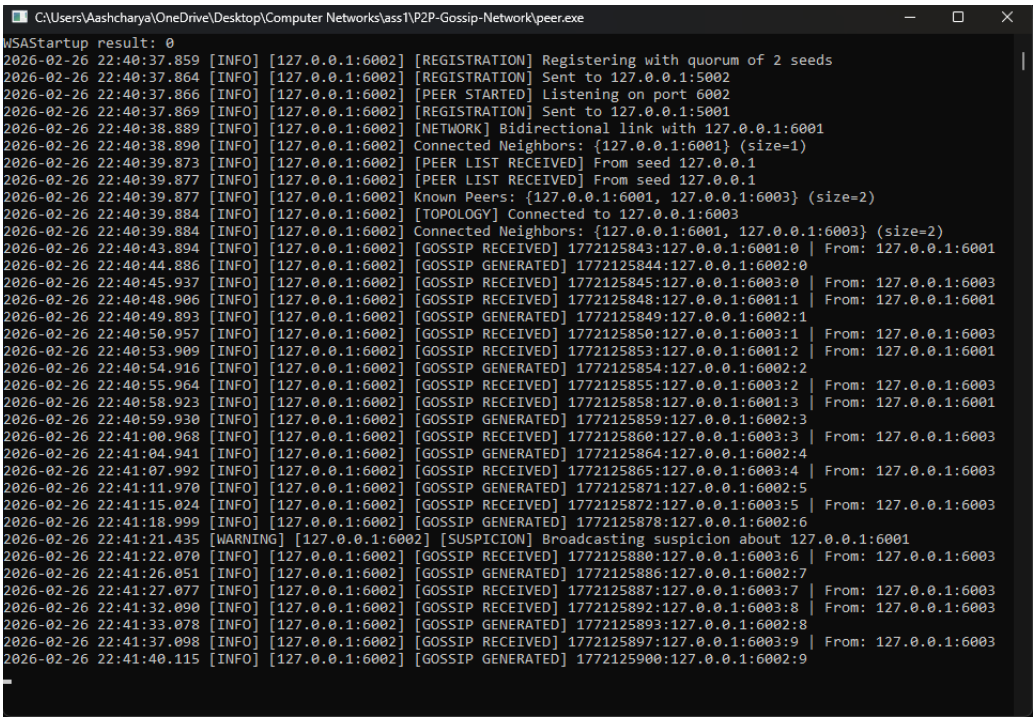
Figure 10: Terminal output of `python run_all.py`: compilation, seed/peer launch, and interactive command menu with `kill peer_6001` to simulate a node failure.

8.3 Console Output Screenshots



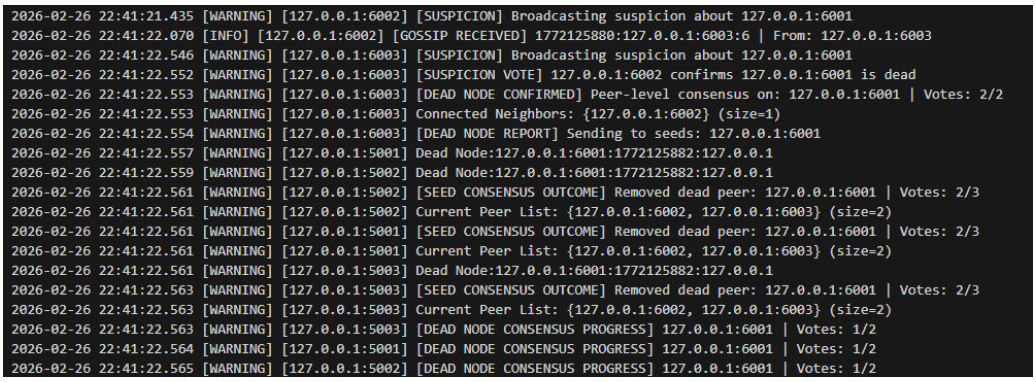
```
C:\Users\Aashcharya\OneDrive\Desktop\Computer Networks\ass1\P2P-Gossip-Network\seed.exe
WSAStartup result: 0
2026-02-26 22:40:34.613 [INFO] [127.0.0.1:5001] [SEED STARTED] Running at 127.0.0.1:5001
2026-02-26 22:40:36.867 [INFO] [127.0.0.1:5001] [CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:40:36.870 [INFO] [127.0.0.1:5001] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:40:36.870 [INFO] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6001} (size=1)
2026-02-26 22:40:37.868 [INFO] [127.0.0.1:5001] [CONSENSUS PROGRESS] 127.0.0.1:6002 | Votes: 1/2
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:5001] [REGISTRATION PROPOSAL] 127.0.0.1:6002
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:5001] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6002 | Votes: 2/3
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6001, 127.0.0.1:6002} (size=2)
2026-02-26 22:40:38.903 [INFO] [127.0.0.1:5001] [REGISTRATION PROPOSAL] 127.0.0.1:6003
2026-02-26 22:40:38.904 [INFO] [127.0.0.1:5001] [CONSENSUS PROGRESS] 127.0.0.1:6003 | Votes: 1/2
2026-02-26 22:40:38.907 [INFO] [127.0.0.1:5001] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6003 | Votes: 2/3
2026-02-26 22:40:38.907 [INFO] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6001, 127.0.0.1:6002, 127.0.0.1:6003} (size=3)
2026-02-26 22:41:04.621 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:41:22.557 [WARNING] [127.0.0.1:5001] Dead Node:127.0.0.1:6001:1772125882:127.0.0.1
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5001] [SEED CONSENSUS OUTCOME] Removed dead peer: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6002, 127.0.0.1:6003} (size=2)
2026-02-26 22:41:22.564 [WARNING] [127.0.0.1:5001] [DEAD NODE CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:41:34.629 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:42:04.632 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:42:34.649 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:43:04.654 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:43:34.666 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:44:04.685 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:44:34.698 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:45:04.709 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
2026-02-26 22:45:34.724 [INFO] [127.0.0.1:5001] [SYNC] Periodic peer-list sync broadcasted to other seeds
```

Figure 11: Seed node console output showing registration proposals, consensus outcomes with vote counts, peer list snapshots, dead-node removal, and periodic sync events.



```
C:\Users\Aashcharya\OneDrive\Desktop\Computer Networks\ass1\P2P-Gossip-Network\peer.exe
WSAStartup result: 0
2026-02-26 22:40:37.859 [INFO] [127.0.0.1:6002] [REGISTRATION] Registering with quorum of 2 seeds
2026-02-26 22:40:37.864 [INFO] [127.0.0.1:6002] [REGISTRATION] Sent to 127.0.0.1:5002
2026-02-26 22:40:37.866 [INFO] [127.0.0.1:6002] [PEER STARTED] Listening on port 6002
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:6002] [REGISTRATION] Sent to 127.0.0.1:5001
2026-02-26 22:40:38.889 [INFO] [127.0.0.1:6002] [NETWORK] Bidirectional link with 127.0.0.1:6001
2026-02-26 22:40:38.890 [INFO] [127.0.0.1:6002] Connected Neighbors: {127.0.0.1:6001} (size=1)
2026-02-26 22:40:39.873 [INFO] [127.0.0.1:6002] [PEER LIST RECEIVED] From seed 127.0.0.1
2026-02-26 22:40:39.877 [INFO] [127.0.0.1:6002] [PEER LIST RECEIVED] From seed 127.0.0.1
2026-02-26 22:40:39.877 [INFO] [127.0.0.1:6002] Known Peers: {127.0.0.1:6001, 127.0.0.1:6003} (size=2)
2026-02-26 22:40:39.884 [INFO] [127.0.0.1:6002] [TOPOLOGY] Connected to 127.0.0.1:6003
2026-02-26 22:40:39.884 [INFO] [127.0.0.1:6002] Connected Neighbors: {127.0.0.1:6001, 127.0.0.1:6003} (size=2)
2026-02-26 22:40:43.894 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125843:127.0.0.1:6001:0 | From: 127.0.0.1:6001
2026-02-26 22:40:44.886 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125844:127.0.0.1:6002:0
2026-02-26 22:40:45.937 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125845:127.0.0.1:6003:0 | From: 127.0.0.1:6003
2026-02-26 22:40:48.906 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125848:127.0.0.1:6001:1 | From: 127.0.0.1:6001
2026-02-26 22:40:49.893 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125849:127.0.0.1:6002:1
2026-02-26 22:40:50.957 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125850:127.0.0.1:6003:1 | From: 127.0.0.1:6003
2026-02-26 22:40:53.909 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125853:127.0.0.1:6001:2 | From: 127.0.0.1:6001
2026-02-26 22:40:54.916 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125854:127.0.0.1:6002:2
2026-02-26 22:40:55.964 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125855:127.0.0.1:6003:2 | From: 127.0.0.1:6003
2026-02-26 22:40:58.923 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125858:127.0.0.1:6001:3 | From: 127.0.0.1:6001
2026-02-26 22:40:59.930 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125859:127.0.0.1:6002:3
2026-02-26 22:41:00.968 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125860:127.0.0.1:6003:3 | From: 127.0.0.1:6003
2026-02-26 22:41:04.941 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125864:127.0.0.1:6002:4
2026-02-26 22:41:07.992 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125865:127.0.0.1:6003:4 | From: 127.0.0.1:6003
2026-02-26 22:41:11.970 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125871:127.0.0.1:6002:5
2026-02-26 22:41:15.024 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125872:127.0.0.1:6003:5 | From: 127.0.0.1:6003
2026-02-26 22:41:18.999 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125878:127.0.0.1:6002:6
2026-02-26 22:41:21.435 [WARNING] [127.0.0.1:6002] [SUSPICION] Broadcasting suspicion about 127.0.0.1:6001
2026-02-26 22:41:22.070 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125880:127.0.0.1:6003:6 | From: 127.0.0.1:6003
2026-02-26 22:41:26.051 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125886:127.0.0.1:6002:7
2026-02-26 22:41:27.077 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125887:127.0.0.1:6003:7 | From: 127.0.0.1:6003
2026-02-26 22:41:32.090 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125892:127.0.0.1:6003:8 | From: 127.0.0.1:6003
2026-02-26 22:41:33.078 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125893:127.0.0.1:6002:8
2026-02-26 22:41:37.098 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125897:127.0.0.1:6003:9 | From: 127.0.0.1:6003
2026-02-26 22:41:40.115 [INFO] [127.0.0.1:6002] [GOSSIP GENERATED] 1772125900:127.0.0.1:6002:9
```

Figure 12: Peer node console output showing registration, bidirectional link formation, peer list received, topology construction, gossip generation/reception, and suspicion events.



```
2026-02-26 22:41:21.435 [WARNING] [127.0.0.1:6002] [SUSPICION] Broadcasting suspicion about 127.0.0.1:6001
2026-02-26 22:41:22.070 [INFO] [127.0.0.1:6002] [GOSSIP RECEIVED] 1772125880:127.0.0.1:6003:6 | From: 127.0.0.1:6003
2026-02-26 22:41:22.546 [WARNING] [127.0.0.1:6003] [SUSPICION] Broadcasting suspicion about 127.0.0.1:6001
2026-02-26 22:41:22.552 [WARNING] [127.0.0.1:6003] [SUSPICION VOTE] 127.0.0.1:6002 confirms 127.0.0.1:6001 is dead
2026-02-26 22:41:22.553 [WARNING] [127.0.0.1:6003] [DEAD NODE CONFIRMED] Peer-level consensus on: 127.0.0.1:6001 | Votes: 2/2
2026-02-26 22:41:22.553 [WARNING] [127.0.0.1:6003] Connected Neighbors: {127.0.0.1:6002} (size=1)
2026-02-26 22:41:22.554 [WARNING] [127.0.0.1:6003] [DEAD NODE REPORT] Sending to seeds: 127.0.0.1:6001
2026-02-26 22:41:22.557 [WARNING] [127.0.0.1:5001] Dead Node:127.0.0.1:6001:1772125882:127.0.0.1
2026-02-26 22:41:22.559 [WARNING] [127.0.0.1:5002] Dead Node:127.0.0.1:6001:1772125882:127.0.0.1
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5002] [SEED CONSENSUS OUTCOME] Removed dead peer: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5002] Current Peer List: {127.0.0.1:6002, 127.0.0.1:6003} (size=2)
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5001] [SEED CONSENSUS OUTCOME] Removed dead peer: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6002, 127.0.0.1:6003} (size=2)
2026-02-26 22:41:22.561 [WARNING] [127.0.0.1:5003] Dead Node:127.0.0.1:6001:1772125882:127.0.0.1
2026-02-26 22:41:22.563 [WARNING] [127.0.0.1:5003] [SEED CONSENSUS OUTCOME] Removed dead peer: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:41:22.563 [WARNING] [127.0.0.1:5003] Current Peer List: {127.0.0.1:6002, 127.0.0.1:6003} (size=2)
2026-02-26 22:41:22.563 [WARNING] [127.0.0.1:5003] [DEAD NODE CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:41:22.564 [WARNING] [127.0.0.1:5001] [DEAD NODE CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:41:22.565 [WARNING] [127.0.0.1:5002] [DEAD NODE CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
```

Figure 13: Dead-node test output: after `kill peer_6001`, surviving peers broadcast suspicion, reach peer-level consensus, report to seeds, and seeds achieve quorum to remove the dead peer from the global peer list.

9 How to Compile and Run

9.1 Prerequisites

- **Windows 10/11** (required for Winsock2).
- **MinGW-w64** with `g++` supporting C++17.
- **Python 3.x** (optional, for the `run_all.py` automation script).
- A modern web browser (for the visualizer).

9.2 Quick Start (Automated)

```python
# Automated: compiles + launches 3 seeds + 3 peers
python run_all.py
```

Listing 4: Starting the full network

9.3 Manual Compilation

```bash
# Compile seed node
g++ seed.cpp common.cpp -lws2_32 -std=c++17 ^
    -static-libgcc -static-libstdc++ ^
    -Wl,-Bstatic -lpthread -Wl,-Bdynamic ^
    -o seed.exe

# Compile peer node
g++ peer.cpp common.cpp -lws2_32 -std=c++17 ^
    -static-libgcc -static-libstdc++ ^
    -Wl,-Bstatic -lpthread -Wl,-Bdynamic ^
    -o peer.exe
```

Listing 5: Compiling seed and peer binaries

9.4 Starting Individual Nodes

```
# Seed nodes (open 3 separate terminals)
.\seed.exe 127.0.0.1 5001
.\seed.exe 127.0.0.1 5002
.\seed.exe 127.0.0.1 5003

# Peer nodes (open 3 separate terminals, stagger by ~1s)
.\peer.exe 127.0.0.1 6001
.\peer.exe 127.0.0.1 6002
.\peer.exe 127.0.0.1 6003
```

Listing 6: Starting individual nodes

9.5 Network Replay Visualizer

```
# Open in any web browser
start visualizer.html
# Upload outputfile.txt and click Play
```

Listing 7: Using the visualizer

The visualizer provides an interactive HTML5 dashboard where you can upload `outputfile.txt`, then replay the network's evolution step by step with adjustable speed.

9.6 Output Files

- `outputfile.txt` — combined log of the latest run (project root). All nodes append to this file concurrently.
- `config.txt` — seed addresses (`IP:Port` per line), read by both seed and peer binaries at startup.

10 GUI and Visualization

An interactive HTML5/JavaScript network replay dashboard (`visualizer.html`) is provided. It parses the generated `outputfile.txt` and replays the network's state chronologically:

10.1 Live Network Dashboard (`visualizer.html`)

- **Topology canvas**: Seeds shown as diamonds (gold), peers as circles (blue), edges drawn in real-time as connections form. Dead nodes turn red; removed nodes turn gray.
- **Stats dashboard**: Live counts of seeds, peers, edges, gossip messages, consensus events, dead node reports, and removals.
- **Event feed**: Color-coded scrolling log of network events with search/filter functionality.

- **Playback controls**: Play/Pause, Step Forward, Reset, and adjustable speed slider for frame-by-frame replay.
- **Gossip animation**: Edges flash green when gossip traverses them; nodes pulse on gossip receipt.
- **Legend bar**: Color key identifying all visual elements (seed, peer, dead, removed, gossip flash, edge).

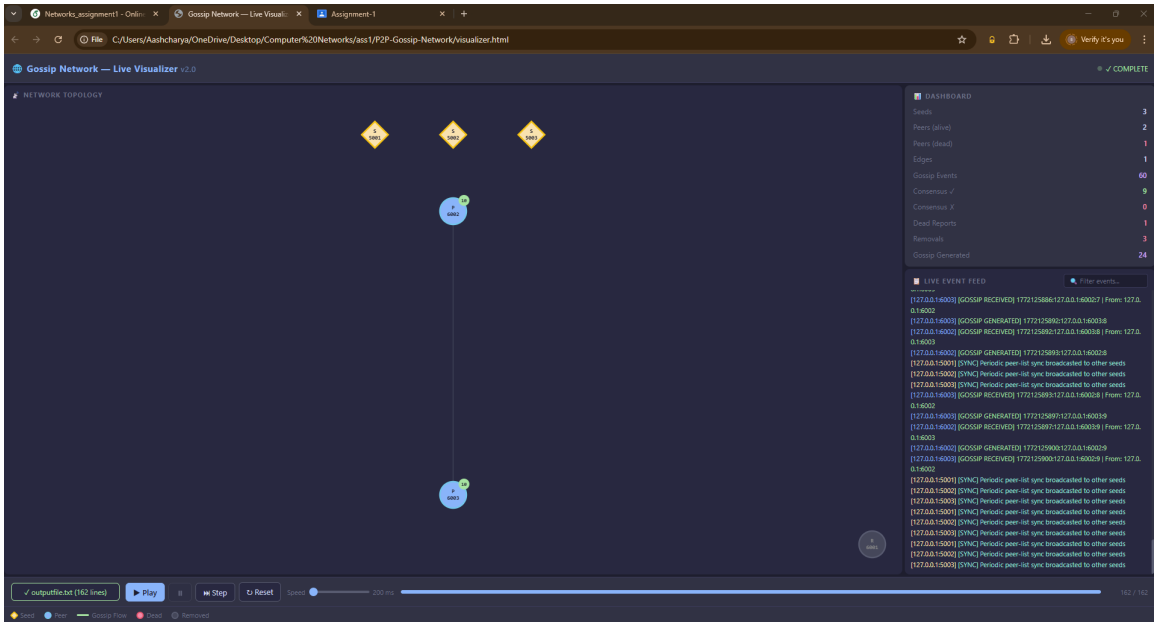
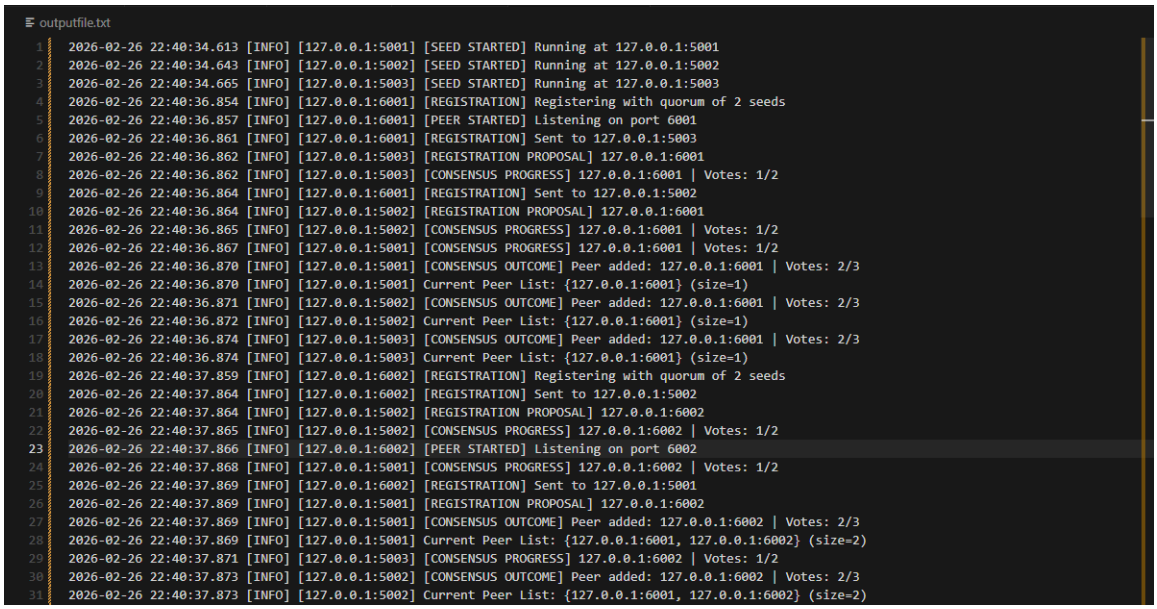


Figure 14: Live network dashboard showing the topology canvas (seeds as gold diamonds, peers as blue circles, dead node as gray), stats panel (gossip events, consensus counts, removals), live event feed, and playback controls.

The visualizer is a single self-contained HTML file with no external dependencies, using the Canvas API for rendering. It features a dark Catppuccin Mocha-inspired theme for readability during demonstrations.

11 Sample Log Output

Below is a representative excerpt from `outputfile.txt` showing the key events across a network session. Figure 15 shows the actual log file opened in the editor.



```
outputfile.txt
2026-02-26 22:40:34.613 [INFO] [127.0.0.1:5001] [SEED STARTED] Running at 127.0.0.1:5001
2026-02-26 22:40:34.643 [INFO] [127.0.0.1:5002] [SEED STARTED] Running at 127.0.0.1:5002
2026-02-26 22:40:34.665 [INFO] [127.0.0.1:5003] [SEED STARTED] Running at 127.0.0.1:5003
2026-02-26 22:40:36.854 [INFO] [127.0.0.1:6001] [REGISTRATION] Registering with quorum of 2 seeds
2026-02-26 22:40:36.857 [INFO] [127.0.0.1:6001] [PEER STARTED] Listening on port 6001
2026-02-26 22:40:36.861 [INFO] [127.0.0.1:6001] [REGISTRATION] Sent to 127.0.0.1:5003
2026-02-26 22:40:36.862 [INFO] [127.0.0.1:5003] [REGISTRATION PROPOSAL] 127.0.0.1:6001
2026-02-26 22:40:36.862 [INFO] [127.0.0.1:5003] [CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:40:36.864 [INFO] [127.0.0.1:6001] [REGISTRATION] Sent to 127.0.0.1:5002
2026-02-26 22:40:36.864 [INFO] [127.0.0.1:5002] [REGISTRATION PROPOSAL] 127.0.0.1:6001
2026-02-26 22:40:36.865 [INFO] [127.0.0.1:5002] [CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:40:36.867 [INFO] [127.0.0.1:5001] [CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
2026-02-26 22:40:36.870 [INFO] [127.0.0.1:5001] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:40:36.870 [INFO] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6001} (size=1)
2026-02-26 22:40:36.871 [INFO] [127.0.0.1:5002] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:40:36.872 [INFO] [127.0.0.1:5002] Current Peer List: {127.0.0.1:6001} (size=1)
2026-02-26 22:40:36.874 [INFO] [127.0.0.1:5003] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6001 | Votes: 2/3
2026-02-26 22:40:36.874 [INFO] [127.0.0.1:5003] Current Peer List: {127.0.0.1:6001} (size=1)
2026-02-26 22:40:37.859 [INFO] [127.0.0.1:6002] [REGISTRATION] Registering with quorum of 2 seeds
2026-02-26 22:40:37.864 [INFO] [127.0.0.1:6002] [REGISTRATION] Sent to 127.0.0.1:5002
2026-02-26 22:40:37.864 [INFO] [127.0.0.1:5002] [REGISTRATION PROPOSAL] 127.0.0.1:6002
2026-02-26 22:40:37.865 [INFO] [127.0.0.1:5002] [CONSENSUS PROGRESS] 127.0.0.1:6002 | Votes: 1/2
2026-02-26 22:40:37.866 [INFO] [127.0.0.1:6002] [PEER STARTED] Listening on port 6002
2026-02-26 22:40:37.868 [INFO] [127.0.0.1:5001] [CONSENSUS PROGRESS] 127.0.0.1:6002 | Votes: 1/2
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:6002] [REGISTRATION] Sent to 127.0.0.1:5001
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:5001] [REGISTRATION PROPOSAL] 127.0.0.1:6002
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:5001] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6002 | Votes: 2/3
2026-02-26 22:40:37.869 [INFO] [127.0.0.1:5001] Current Peer List: {127.0.0.1:6001, 127.0.0.1:6002} (size=2)
2026-02-26 22:40:37.871 [INFO] [127.0.0.1:5003] [CONSENSUS PROGRESS] 127.0.0.1:6002 | Votes: 1/2
2026-02-26 22:40:37.873 [INFO] [127.0.0.1:5002] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6002 | Votes: 2/3
2026-02-26 22:40:37.873 [INFO] [127.0.0.1:5002] Current Peer List: {127.0.0.1:6001, 127.0.0.1:6002} (size=2)
```

Figure 15: Head of `outputfile.txt` in the editor: seed startup, peer registration with consensus progress/outcome, peer list snapshots, and topology events—all with millisecond-precision timestamps and log levels.

```
[127.0.0.1:5001] [SEED STARTED] Running at 127.0.0.1:5001
[127.0.0.1:5002] [SEED STARTED] Running at 127.0.0.1:5002
[127.0.0.1:5003] [SEED STARTED] Running at 127.0.0.1:5003

[127.0.0.1:6001] [REGISTRATION] Registering with quorum of 2 seeds
[127.0.0.1:5002] [REGISTRATION PROPOSAL] 127.0.0.1:6001
[127.0.0.1:5002] [CONSENSUS PROGRESS] 127.0.0.1:6001 | Votes: 1/2
[127.0.0.1:5001] [CONSENSUS OUTCOME] Peer added: 127.0.0.1:6001
    | Votes: 2/3
[127.0.0.1:6001] [PEER STARTED] Listening on port 6001

[127.0.0.1:6001] [PEER LIST RECEIVED] From seed 127.0.0.1
[127.0.0.1:6001] [TOPOLOGY] Connected to 127.0.0.1:6002
[127.0.0.1:6002] [NETWORK] Bidirectional link with 127.0.0.1:6001

[127.0.0.1:6001] [GOSSIP GENERATED] 1772104070:127.0.0.1:6001:0
[127.0.0.1:6002] [GOSSIP RECEIVED] 1772104070:127.0.0.1:6001:0
    | From: 127.0.0.1:6001
[127.0.0.1:6003] [GOSSIP RECEIVED] 1772104070:127.0.0.1:6001:0
    | From: 127.0.0.1:6001

[127.0.0.1:6002] [SUSPICION] Broadcasting suspicion about
    127.0.0.1:6001
[127.0.0.1:6003] [SUSPICION VOTE] 127.0.0.1:6002 confirms
    127.0.0.1:6001 is dead
[127.0.0.1:6003] [DEAD NODE CONFIRMED] Peer-level consensus on:
    127.0.0.1:6001 | Votes: 2/2
[127.0.0.1:6003] [DEAD NODE REPORT] Sending to seeds:
    127.0.0.1:6001

[127.0.0.1:5001] Dead Node:127.0.0.1:6001:1772104317:127.0.0.1
[127.0.0.1:5002] [SEED CONSENSUS OUTCOME] Removed dead peer:
    127.0.0.1:6001 | Votes: 2/3
```

Listing 8: Sample output from outputfile.txt

Verifying Logs with grep/findstr

```
# Registration consensus:
findstr "CONSENSUS OUTCOME" outputfile.txt

# Gossip messages:
findstr "GOSSIP RECEIVED" outputfile.txt

# Dead-node reports:
findstr "DEAD NODE" outputfile.txt

# Peer lists and topology:
findstr "TOPOLOGY" outputfile.txt
```

12 Design Decisions and Trade-offs

1. **C++ over Python**: We chose C++ for direct control over sockets, threads, and memory. Winsock2 provides low-level access to the Windows networking stack without abstraction overhead, and `std::thread`/`std::mutex` provide POSIX-like concurrency primitives on Windows.

2. **TCP over UDP**: We chose TCP for all communication to guarantee message delivery and ordering. While UDP would reduce overhead for gossip, TCP's reliability simplifies the protocol significantly and prevents message loss during consensus voting.

3. **Custom string protocol over JSON**: Using simple `TYPE:KEY;VALUE` string encoding avoids the need for a JSON parsing library, keeping the project dependency-free. The overhead is minimal for the message sizes involved (< 1 KB per message), and the format is human-readable for debugging.

4. **Pure C++ SHA-256**: Implementing SHA-256 from scratch (166 lines in `common.cpp`) eliminates the need for OpenSSL or any external cryptographic library, making the project fully self-contained and easy to compile on any MinGW installation.

5. **Dynamic preferential attachment over static Zipf**: Rather than assigning probabilities from a shuffled list (Zipf), we query each candidate's *actual live degree* from the network. This produces a genuine Barabási–Albert power-law topology that adapts to the network's current state, not just the order in which peers are enumerated.

6. **Bidirectional connections**: When peer A connects to peer B via `CONNECT_REQ`, peer B also adds A to its neighbor list. This ensures symmetric topology—if A can gossip to B, B can also gossip to A.

7. **Suspicion threshold of 3**: Requires three consecutive TCP probe failures before entering the suspicion phase. This avoids false positives from transient network issues or briefly unresponsive processes.

8. **Seed-to-seed periodic sync**: Every 30 s, each seed broadcasts its peer list to all other seeds via `TYPE:SYNC_LIST`. This ensures eventual consistency even if initial vote messages were lost due to transient failures or race conditions.

13 Potential Improvements

- **Raft/Paxos for seed consensus**: Replace the simple majority-vote with Raft to handle seed failures gracefully with leader election and log replication.
- **Anti-entropy protocol**: Add periodic full-state synchronization to repair inconsistencies in the Message List across peers.
- **Adaptive suspicion thresholds**: Dynamically adjust TCP probe timeouts based on observed network latency.
- **TLS encryption**: Add TLS/SSL for all TCP connections to prevent eavesdropping and message tampering.
- **Graceful leave protocol**: Implement a mechanism where a departing peer notifies its neighbors and seeds to avoid unnecessary failure detection cycles.
- **Cross-platform support**: Port from Winsock2 to POSIX sockets (`<sys/socket.h>`) to support Linux/macOS deployment.
- **Automated test suite**: Develop a scripted test harness that programmatically verifies all protocol invariants (registration consensus, gossip coverage, failure detection) without manual intervention.
- **Persistent message buffering**: Use framed message lengths instead of a fixed 4096-byte buffer to support arbitrarily large messages and handle partial TCP reads.

14 Conclusion

We have designed and implemented a fully functional gossip-based P2P network that satisfies all assignment requirements:

- **Consensus-based registration**: Peers register only after seed quorum approval ($\lfloor n/2 \rfloor + 1$).
- **Power-law overlay**: Dynamic preferential attachment produces heterogeneous degree distributions following the Barabási–Albert model.
- **Gossip with dedup**: Pure C++ SHA-256 hashing ensures each message traverses each link at most once; 10-message cap per peer with 5 s intervals.
- **Two-level failure detection**: Peer-level majority + seed-level quorum prevents false accusations and ensures safe removal.
- **Comprehensive logging**: All required events logged to console and shared `outputfile.txt` with millisecond-precision timestamps and severity levels.

- **Seed synchronization**: Periodic peer-list broadcast between seeds ensures eventual consistency.
- **Network visualization**: Interactive HTML5 dashboard for replaying and inspecting network state evolution.

The system is implemented entirely in C++17 using only the Windows standard libraries (Winsock2, `<thread>`, `<mutex>`, `<chrono>`) with zero external dependencies, making it compilable with a basic MinGW-w64 installation.

References

[1] Beej's Guide to Network Programming, https://beej.us/guide/bgnet/html/split/system-calls-or-bust.html#socket

[2] A. Demers et al., "Epidemic Algorithms for Replicated Database Maintenance," *Proc. PODC*, 1987.

[3] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm (Raft)," *Proc. USENIX ATC*, 2014.

[4] A.-L. Barabási and R. Albert, "Emergence of Scaling in Random Networks," *Science*, vol. 286, no. 5439, 1999.

[5] Microsoft Winsock2 Documentation, https://learn.microsoft.com/en-us/windows/win32/winsock/windows-sockets-start-page-2

[6] C++ Reference — `<thread>` library, https://en.cppreference.com/w/cpp/thread/thread

[7] C++ Reference — `<mutex>` library, https://en.cppreference.com/w/cpp/thread/mutex

[8] NIST FIPS 180-4 — Secure Hash Standard (SHA-256), https://csrc.nist.gov/publications/detail/fips/180/4/final